SC4003 Assignment 1 Report

March 4, 2026

1 Problem setup

The base maze used in Part 1 is shown in Figure 1. The discount factor is fixed at 0.99, exactly as required by the assignment.

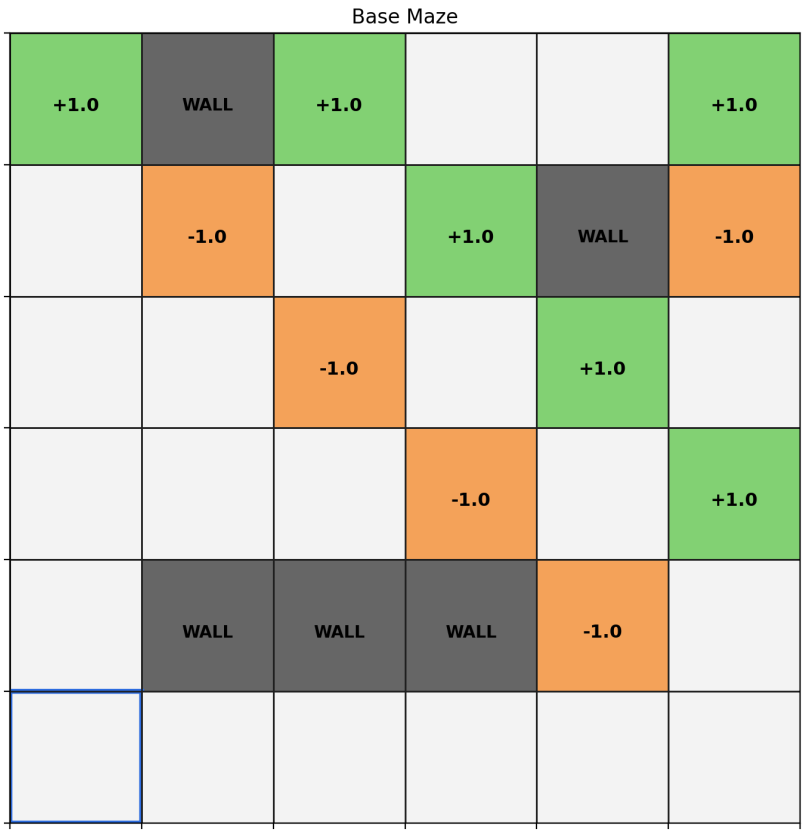

Figure 1: Assignment maze used for Part 1. Green cells have reward +1, brown cells have reward -1, white cells have reward -0.05, and gray cells are walls.

2 Using method of value iteration for Part 1

2.1 Descriptions of implemented solutions

The assignment maze is a 6x6 grid with 5 walls, 6 green reward cells (+1), 5 brown reward cells (-1), and white cells with reward -0.05. The agent has four actions: up, right, down, and left. The intended move happens with probability 0.8, and the two perpendicular moves each happen with probability 0.1. If a move hits a wall or the boundary, the agent stays in the same state. A

crucial detail from the assignment is that there are no terminal states, so green and brown cells remain revisitable. Value Iteration updates every non-wall state with the Bellman optimality equation $U_{k+1}(s) = \max_a \sum_{s'} P(s' \mid s, a)\,[R(s, a, s') + \gamma U_k(s')]$ until the change between two consecutive sweeps is below the stopping threshold.

2.2 Plot of optimal policy

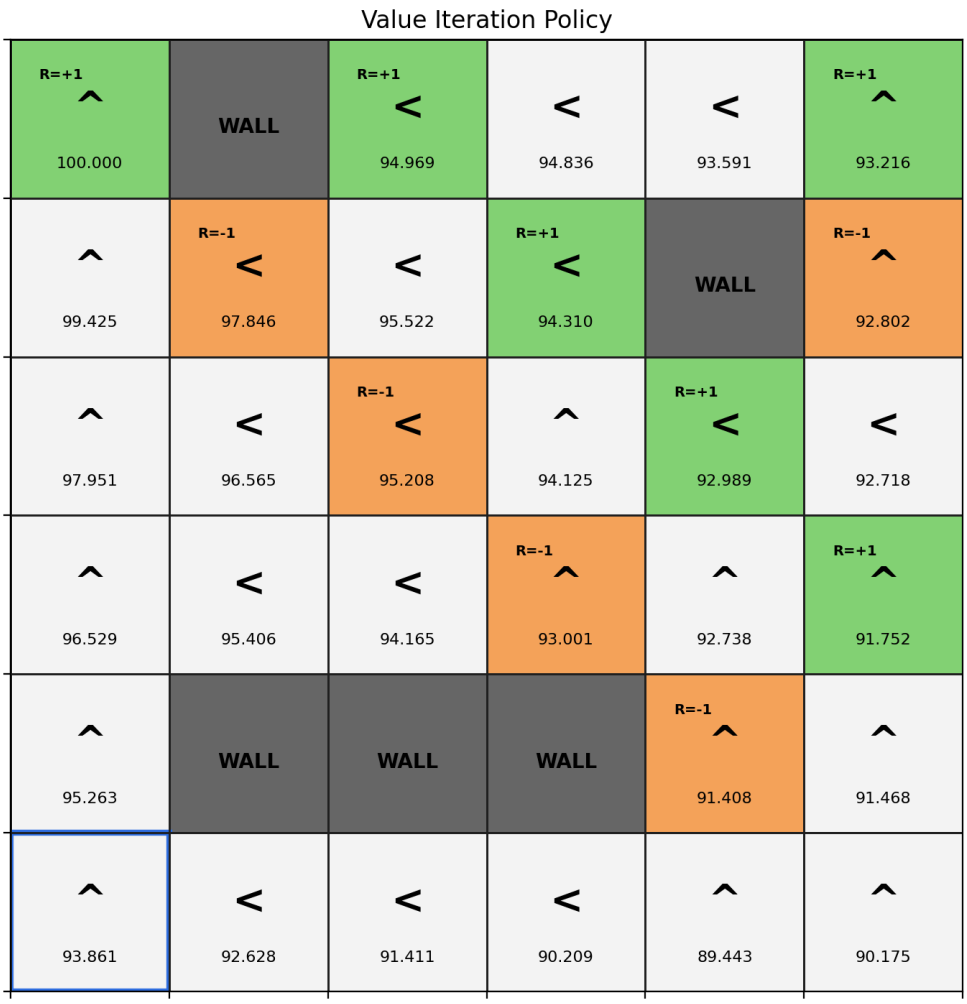

Figure 2: Optimal policy produced by Value Iteration.

The policy points toward repeatedly collecting high reward while avoiding paths that make accidental moves into brown cells more likely. This matches the assignment requirement to display the optimal policy for all non-wall states.

2.3 Utilities of all states

These utilities are much larger than the immediate rewards because the assignment uses an infinite-horizon discounted process with no terminal states. A state is valuable not only for its next reward, but also for how well it can lead the agent back to green cells repeatedly.

2.4 Plot of utility estimates as a function of the number of iterations

The curves flatten out near the end of the plot, which is the practical sign that the utility estimates have converged.

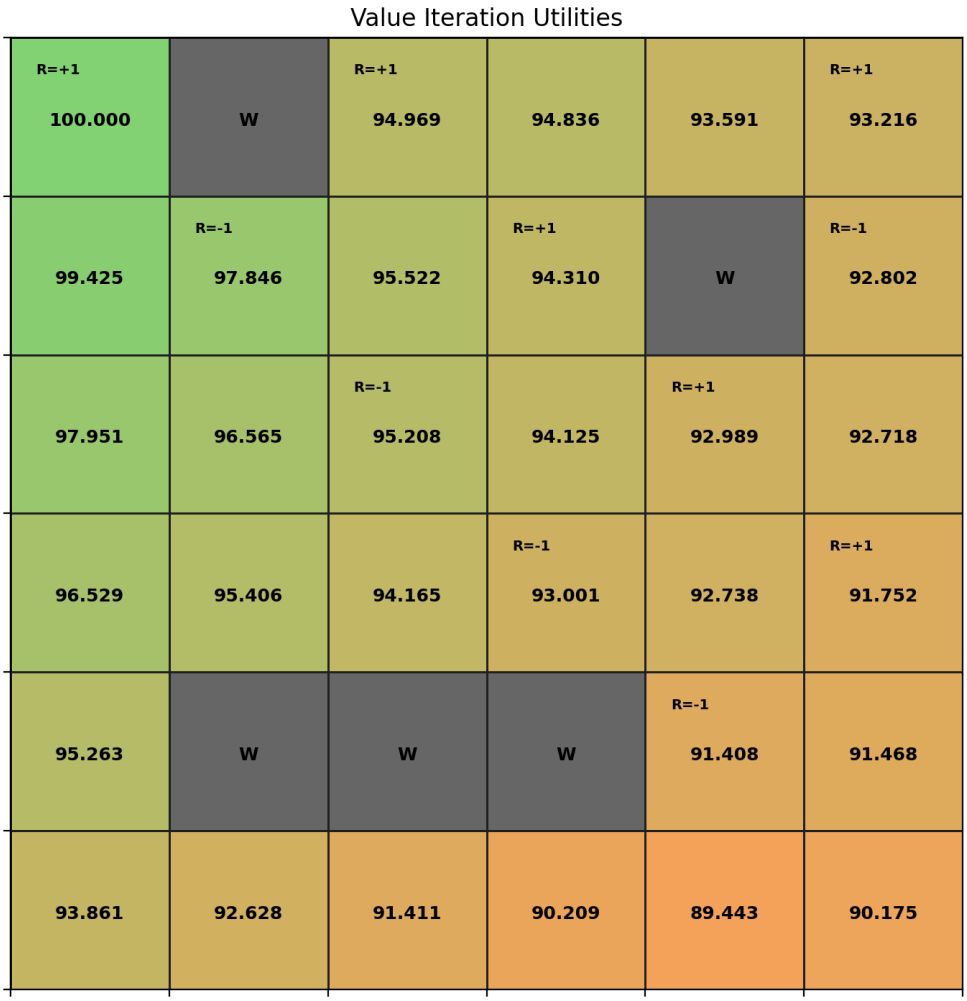

Figure 3: Utility heatmap for Value Iteration. Each number is the utility of a non-wall state. Green and brown reward cells are revisitable states, not terminal states.

3 Using method of policy iteration for Part 1

3.1 Descriptions of implemented solutions

Policy Iteration uses the same MDP model as Value Iteration. It starts from an initial policy, repeatedly evaluates the current policy, and then improves it greedily using the updated utilities. Policy evaluation is implemented by iterative sweeps using $U^\pi(s) = \sum_{s'} P(s' \mid s, \pi(s)) \left[R(s, \pi(s), s') + \gamma U^\pi(s')\right]$, and policy improvement replaces the current action with the action that gives the highest expected return. Because there are no terminal states, the policy is defined for every non-wall cell, including green and brown reward cells.

3.2 Plot of optimal policy

The final policy produced by Policy Iteration matches the one from Value Iteration. This agreement is expected because both algorithms solve the same discounted MDP.

3.3 Utilities of all states

The utility magnitudes are almost identical to those from Value Iteration, which confirms that the two methods converged to the same solution for Part 1.

99.999900	WALL	94.969143	94.836046	93.591206	93.215857
99.424724	97.846025	95.521592	94.310269	WALL	92.801693
97.951464	96.564528	95.208038	94.125258	92.989195	92.718301
96.529158	95.406194	94.164595	93.000559	92.737989	91.752446
95.263207	WALL	WALL	WALL	91.408194	91.467645
93.860878	92.628197	91.410887	90.208755	89.443234	90.174532

Table 1: Utility values for Value Iteration.

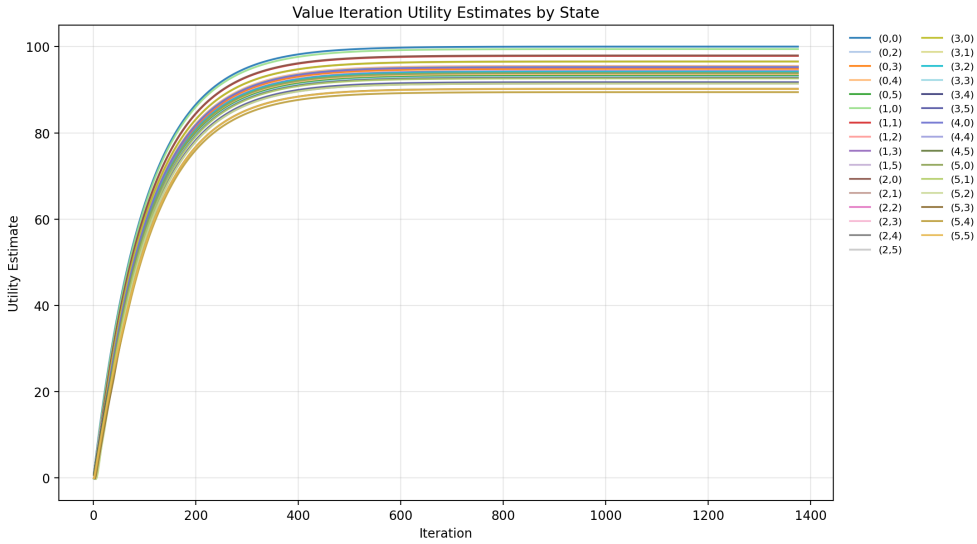

Figure 4: Utility estimate of every non-wall state versus iteration for Value Iteration. Total iterations: 1375. Runtime: 307.915 ms.

3.4 Plot of utility estimates as a function of the number of iterations

The curves flatten out near the end of the plot, which is the practical sign that the utility estimates have converged.

4 Source code for Part 1

- Environment model: `src/gridworld.py`
- Value Iteration: `src/value_iteration.py`
- Policy Iteration: `src/policy_iteration.py`
- Visualization: `src/visualization.py`
- CLI entrypoint: `main.py`

The code follows the report structure directly: the environment model defines the MDP, the two algorithm files implement Value Iteration and Policy Iteration, and the plotting/report code exports the policy plots, utility tables, and convergence figures required in the assignment.

5 Part 2 questions

5.1 Complicated maze design

For Part 2, I designed a family of larger mazes instead of changing only one map manually. Starting from size 7 and increasing to size 17, each generated maze contains more white states,

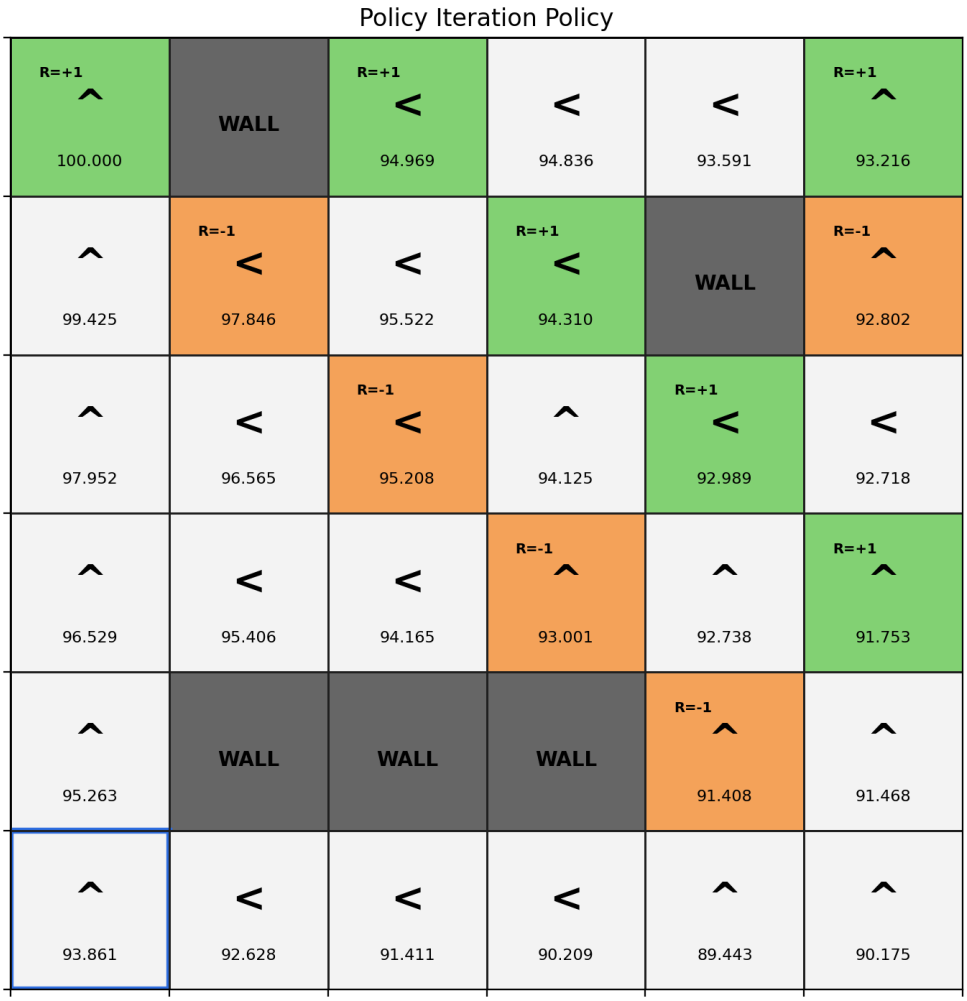

Figure 5: Optimal policy produced by Policy Iteration.

more walls, and more positive/negative reward cells. The same transition model as Part 1 is kept, and the random seed is fixed at 42 so that the experiments are reproducible.

5.2 How do the number of states and the complexity of the environment affect convergence?

Table 3 and the plots below show a clear trend: when the maze gets larger, both algorithms need more computation before the utilities stabilize. Value Iteration usually needs more repeated Bellman sweeps over the whole state space. Policy Iteration often reaches a stable policy with a different update pattern, but its evaluation stage is more expensive per round, so its runtime is not automatically lower.

5.3 How complex can the environment be while still learning the right policy?

In this implementation, both algorithms still converged and produced the same final policy for every tested maze size from 7 to 17. Therefore, within the tested range, the largest environment that still learned a matching optimal policy was the 17x17 maze. The policy mismatch column in Table 3 is zero for every run, which is the main evidence used here.

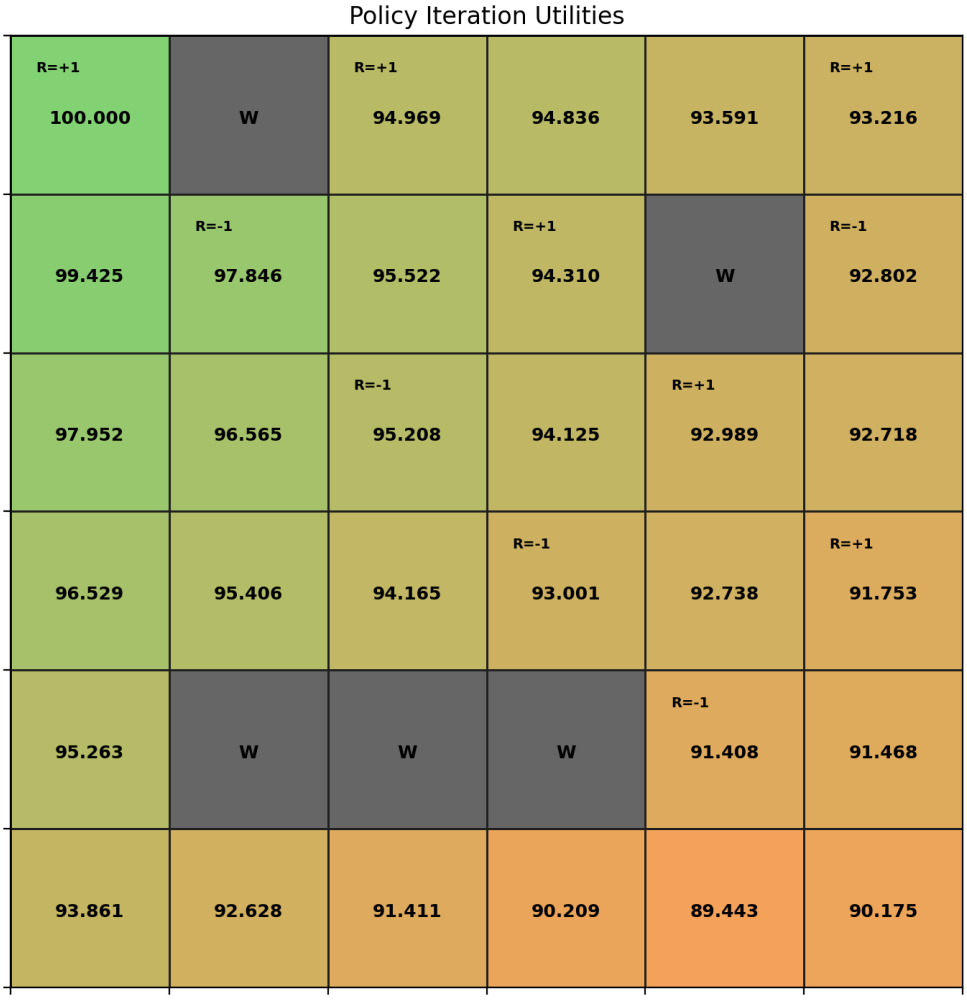

Figure 6: Utility heatmap for Policy Iteration. Each number is the utility of a non-wall state. Green and brown reward cells are revisitable states, not terminal states.

100.000000	WALL	94.969243	94.836145	93.591306	93.215955
99.424824	97.846125	95.521692	94.310369	WALL	92.801791
97.951564	96.564628	95.208137	94.125358	92.989294	92.718400
96.529258	95.406293	94.164695	93.000659	92.738088	91.752546
95.263307	WALL	WALL	WALL	91.408293	91.467744
93.860977	92.628297	91.410986	90.208854	89.443333	90.174631

Table 2: Utility values for Policy Iteration.

Size	VI Iter	PI Iter	VI Runtime	PI Runtime	VI Mean Utility	Mismatch
7	1375	2505	376.27	181.199	92.407394	0
9	1361	3402	687.324	442.225	78.909978	0
11	1361	2830	1023.675	550.358	75.980002	0
13	1375	2936	1474.54	844.812	85.823608	0
15	1361	5102	1939.654	1958.552	74.638951	0
17	1361	5531	2558.762	2700.895	73.871966	0

Table 3: Part 2 comparison across generated mazes.

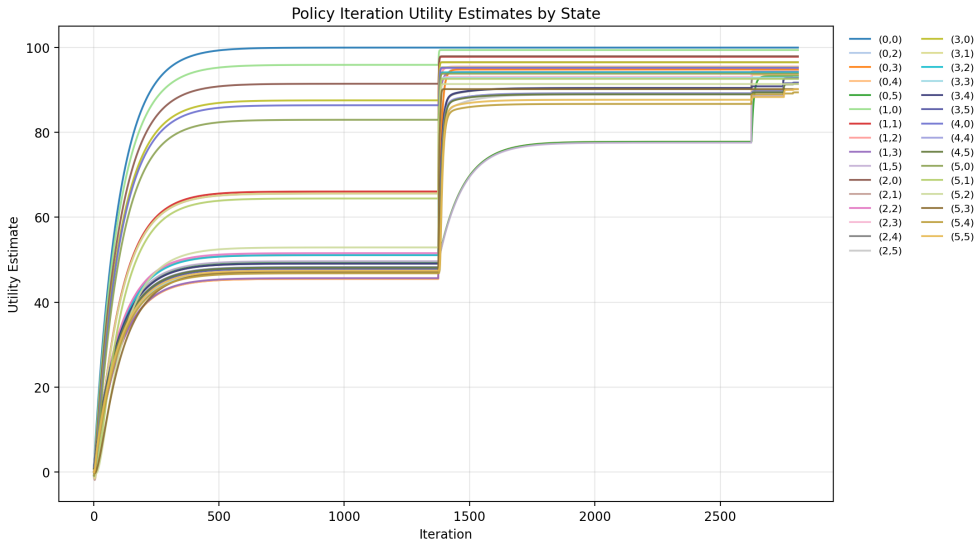

Figure 7: Utility estimate of every non-wall state versus iteration for Policy Iteration. Total iterations: 2810. Runtime: 156.465 ms.

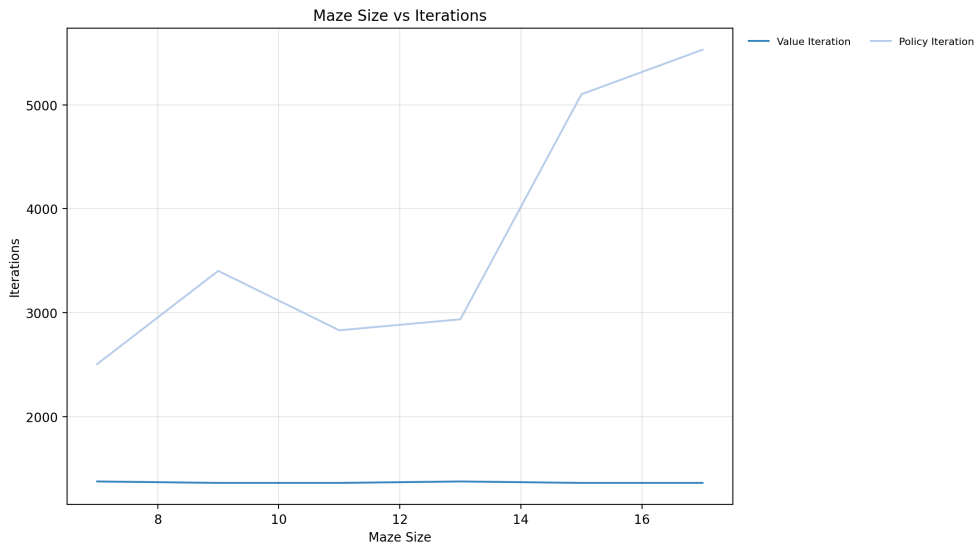

Figure 8: Iterations required by each method as maze size increases.

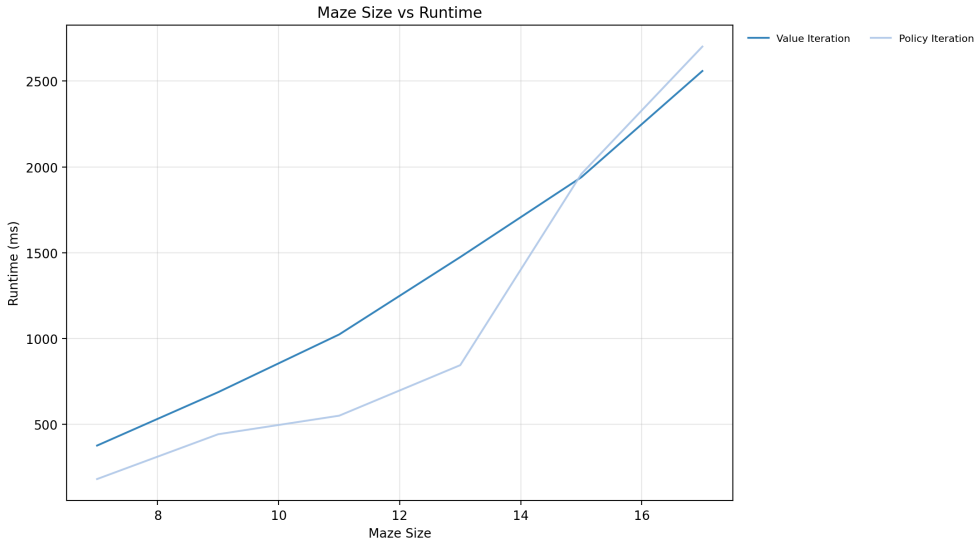

Figure 9: Runtime comparison as maze size increases.

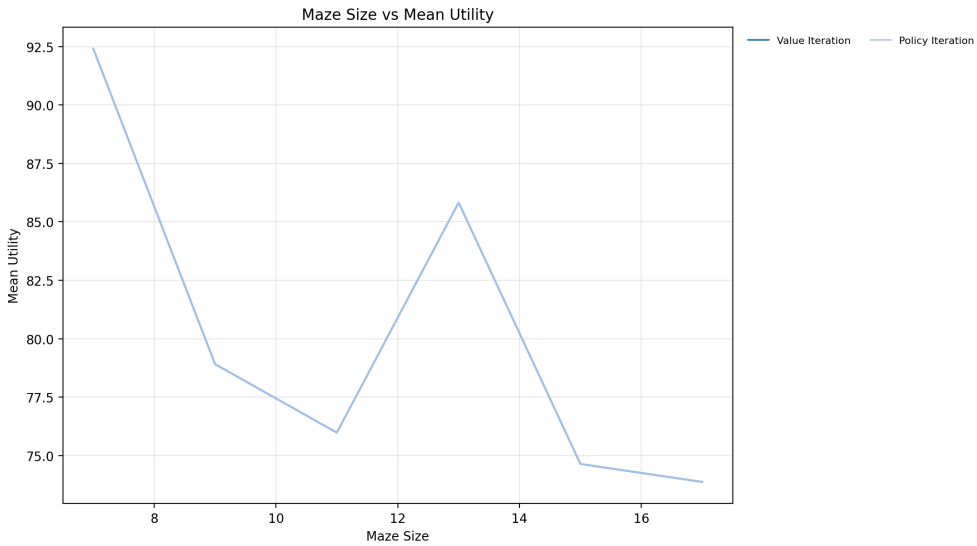

Figure 10: Mean utility comparison as maze size increases.